A PROJECT REPORT ON

Contextual Word Prediction Using BERT-Based Masked Language Modelling

Natural Language Processing

Assignment 1

Topic 4: Fill-Mask

Chapter 2: Application Topics

Abstract

Predicting a missing word requires an understanding of the grammatical and semantic information provided by its surrounding context. This project examines the fill-mask task using a pretrained BERT masked language model. A sentence containing a single mask token is supplied to the system, which returns ranked vocabulary candidates and their completed sequences. The proposed application is a vocabulary assistance tool for short English sentences. Its architecture comprises input validation, tokenization, contextual representation, masked-position prediction and output interpretation. A Python implementation is presented using the google-bert/bert-base-uncased checkpoint and the Hugging Face Transformers library. The methodology defines how vocabulary logits are converted into probabilities and how the highest-ranked candidates are selected. Evaluation is based on top-1 and top-k token recovery, supplemented by human assessment of grammatical and semantic acceptability. The design recognizes that a model token may represent a subword rather than a complete word and that several completions may be valid for the same sentence. The project provides a practical framework for contextual token prediction and identifies the limitations of applying a single-mask model to ambiguous sentences and multi-token expressions.

Keywords: masked language modelling; BERT; fill-mask; contextual prediction; NLP

Index

Abstract 2

1. Introduction 4

2. Problem Statement 5

3. Design 6

4. Proposed Methodology 7

5. Implementation 9

6. Result 11

7. Reference 13

1. Introduction

Language comprehension depends on relationships between words rather than on isolated vocabulary. In a sentence completion exercise, the missing item must be consistent with the grammatical structure and the meaning of the surrounding text. For example, "The student opened the [MASK] to read" constrains the missing position differently from "The mechanic opened the [MASK] to inspect the engine." A useful prediction system must therefore consider the complete context.

Masked language modelling is a learning task in which selected tokens are hidden and the model learns to predict them from the remaining sequence. BERT, introduced by Devlin et al., uses bidirectional contextual representations that condition on the words before and after a position [1]. A pretrained BERT model with a masked language modelling head can consequently be used to rank candidate replacements at a specified gap.

1.1 Fill-mask as an application

The fill-mask task applies this capability to an explicitly masked input. The system receives a sentence, locates the special mask token and returns likely replacements. It differs from unrestricted text generation because the target position is already specified. The present project uses the task to design a vocabulary assistance tool that presents several alternatives, allowing a learner to assess how each replacement affects the sentence.

1.2 Selected checkpoint and token representation

The selected checkpoint is google-bert/bert-base-uncased. It is pretrained on English text and treats uppercase and lowercase variants as equivalent [2]. Its tokenizer represents text using WordPiece units. Consequently, a single model token may correspond to a complete word or a subword fragment. This distinction matters when describing the application: the implementation predicts one token at the masked position, not an arbitrary phrase of unspecified length.

The proposed tool retains candidate alternatives rather than assuming that the highest-ranked token is the only acceptable completion. Contextual plausibility, factual correctness and recovery of an original hidden token are different criteria. They must be considered separately when evaluating the system.

2. Problem Statement

The problem is to predict suitable replacements for one masked position in a short English sentence using both the preceding and following context. The system must identify the mask reliably, reject unsupported input and return a ranked candidate list with completed sentences. Predictions should be interpretable without suggesting that model probability is a guarantee of correctness.

2.1 Objectives

The project aims to implement contextual token prediction with a pretrained masked language model, validate the number and position of mask tokens, select the top-k vocabulary candidates and display their completed sequences. A further objective is to define an evaluation procedure that measures both recovery of the original token and the acceptability of alternative completions.

2.2 Scope and constraints

The initial implementation accepts one short English sentence containing exactly one [MASK] marker. The number of suggestions is restricted to an integer between 1 and 10. The encoded sentence must fit the model's input limit. Multiple masks, unrestricted phrase generation and multilingual completion are excluded from this version. These restrictions keep the prediction target and output interpretation clearly defined.

Table 1. Functional requirements of the fill-mask application.

Requirement	Specification
Input	A non-empty sentence with exactly one [MASK] marker
Parameter	top-k value from 1 to 10
Processing	Prediction at the masked token position
Output	Candidate tokens, scores and completed sequences
Constraint	Input must fit the tokenizer sequence limit

3. Design

3.1 System architecture

The system separates input checking from contextual prediction. The input module obtains the sentence and requested number of suggestions. Validation verifies the mask count and token length. The tokenizer maps the sentence to token identifiers and adds the model's special tokens. BERT produces a contextual representation at each position, and the masked language modelling head maps the masked representation to vocabulary logits. The post-processing module selects candidate tokens and constructs their completed sequences.

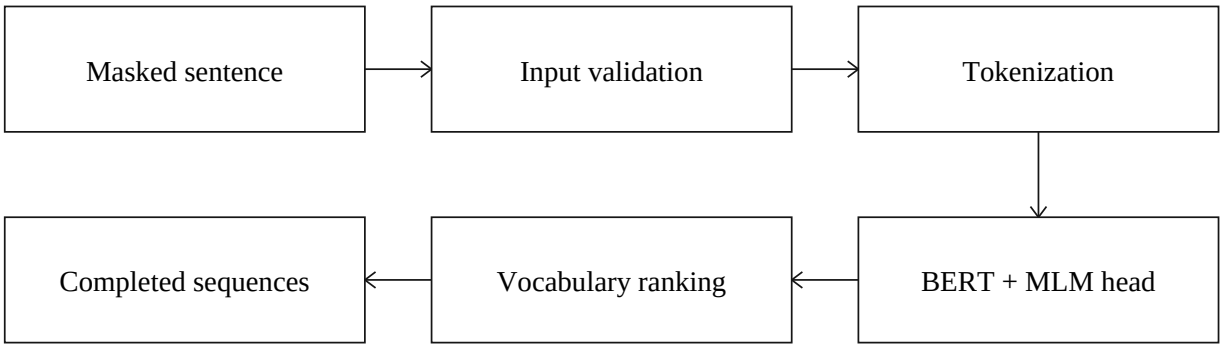


Figure 1. Processing architecture for contextual token prediction.

3.2 Contextual representation

Transformer attention allows representations to incorporate information from other positions in the sequence [5]. In this application, the representation at the mask position uses evidence from both sides of the gap. The output vocabulary distribution is calculated for that position. The remaining sentence is retained as context rather than independently generated.

3.3 Output structure

Each candidate record contains a token identifier, its decoded form, a score and a completed sequence [4]. These fields serve different purposes: the identifier establishes the exact vocabulary item, the decoded form supports display and the completed sequence shows the replacement in context. The original input should also be retained when capitalization and punctuation matter, because the uncased tokenizer may normalize the displayed sequence.

4. Proposed Methodology

4.1 Input preparation

The sentence is entered with [MASK] at the intended prediction position. Surrounding whitespace is removed, but word order and punctuation are preserved. The mask count is checked before tokenization. The requested top-k value is validated, and the encoded length is compared with the tokenizer's maximum sequence length. Rejecting an overlength input avoids silently removing the mask or context through automatic truncation.

4.2 Masked-position inference

The encoded sentence is passed to the pretrained masked language model. Let z_v be the output logit for vocabulary token v at the mask position and let V be the vocabulary. The softmax operation gives the model probability assigned to each candidate [4]:

$$p(v \mid context) = exp(z_v) / \Sigma_{u \in V} exp(z_u) \quad (1)$$

Candidates are ranked by this probability and the k highest values are retained. The distribution is normalized over the full vocabulary, not over the displayed subset. Therefore, the top-k scores need not sum to one. A low total probability for the displayed candidates may indicate that the distribution is spread across many alternatives.

4.3 Algorithm

Step 1: Read the masked sentence and top-k value.
Step 2: Check that the input contains exactly one supported mask.
Step 3: Tokenize the sentence and verify its encoded length.
Step 4: Obtain vocabulary logits at the masked position.
Step 5: Apply softmax and select the k highest-ranked tokens.
Step 6: Decode each candidate and display its completed sequence.

4.4 Evaluation procedure

An evaluation set can be prepared from 30 short sentences with one hidden target token in each. The targets should include nouns, verbs and context-sensitive alternatives. Each removed target must correspond to exactly one tokenizer token; otherwise a single mask would not represent the same recovery task. The original token is retained as the reference answer, while sentences and mask positions are fixed before the experiment.

$$\textit{Top-k accuracy} = \textit{Targets found in the top-k list} / \textit{Number of test sentences} \quad (2)$$

Top-1 accuracy measures exact recovery by the highest-ranked candidate. Top-k accuracy records whether the original target appears anywhere in the candidate list. Exact recovery should be supplemented by human review because a different token may produce a grammatical and semantically acceptable sentence. The review can record grammatical fit and contextual relevance separately. Factual sentences should also be checked against an external reference rather than judged by model confidence alone.

5. Implementation

5.1 Software environment

The implementation uses Python, PyTorch and the Transformers fill-mask pipeline [3]. The checkpoint is specified explicitly, and CPU execution is selected with device=-1. The first run requires internet access to obtain model files. Subsequent runs may use the local cache. Package versions and the exact checkpoint revision should be recorded when conducting an experiment.

Installation: `python -m pip install torch transformers`
Save Listing 1 as `fill_mask.py` and run `python fill_mask.py`.

Listing 1. Core implementation of the fill-mask application.

```
from transformers import pipeline

unmasker = pipeline(
    "fill-mask",
    model="google-bert/bert-base-uncased",
    device=-1
)

text = "The capital of France is [MASK]."
top_k = 5
tokenizer = unmasker.tokenizer

if text.count(tokenizer.mask_token) != 1:
    raise ValueError("Exactly one mask token is required.")
if type(top_k) is not int or not 1 <= top_k <= 10:
    raise ValueError("top_k must be an integer from 1 to 10.")
encoded = tokenizer(text)
if len(encoded["input_ids"]) > tokenizer.model_max_length:
    raise ValueError("Input exceeds the model token limit.")

predictions = unmasker(text, top_k=top_k)

for item in predictions:
    print(item["token_str"], round(item["score"], 4))
    print(item["sequence"])
```

5.2 Implementation details

The tokenizer is taken from the loaded pipeline to ensure that the input uses the mask marker expected by the selected checkpoint. Validation rejects missing or repeated masks, invalid top-k values and sequences exceeding the token limit. The pipeline then returns the prediction list. Candidate scores and completed sequences are printed together so that each suggestion can be examined in its original context.

The task-specific pipeline carries out model execution and post-processing. The application should not renormalize the returned top-k scores because those values are probabilities relative to the full vocabulary. A graphical interface can present the same records in a table, but it should preserve the token identity and make subword predictions visible. Replacing a subword display with an assumed full word would misrepresent the model output.

5.3 Operational considerations

The model object should be reused across multiple requests. Network failures during the initial download and memory limitations are execution issues that require application-level handling. Initial loading time should be excluded from steady-state prediction measurements. The single-mask restriction is enforced even where library versions offer experimental support for more than one mask, since independent masked-position scores do not necessarily define a joint phrase probability.

6. Result

6.1 Qualitative case analysis

Table 2 presents masked sentences and contextually plausible reference completions. These examples illustrate the intended behavior; they are not measured pipeline predictions. The Python listing was checked for syntax, while pretrained-model inference and dataset-level evaluation were not performed during preparation of this report.

Table 2. Illustrative test cases for masked-token prediction.

Masked sentence	Possible completion	Interpretation
The capital of France is [MASK].	paris	A factual completion requiring verification
She drank a glass of [MASK].	water	Several drinks are plausible
The student opened the [MASK] to read.	book	Object inferred from the reading activity
The sun rises in the [MASK].	east	Direction constrained by the sentence

6.2 Interpretation and limitations

The main expected benefit is the use of full-sentence context when suggesting a replacement. However, an ambiguous sentence can support several candidates. In the second example, milk or juice may also be acceptable. A model could therefore fail to recover an original target while still producing a reasonable completion. This explains why exact-match accuracy and human acceptability should be reported as separate measurements.

Tokenization imposes a further limitation. A single mask predicts one vocabulary token, so a multi-token word or phrase cannot be treated as an equivalent one-step task. The uncased model also does not preserve distinctions based solely on capitalization. Predictions may reflect statistical associations in pretraining data, including biased associations, and a fluent completion may be factually incorrect. These limitations should be considered when selecting examples for educational use.

6.3 Conclusion

The proposed application organizes masked-token prediction into a reproducible workflow with explicit input constraints and interpretable candidate records. BERT provides the contextual representations, while the application handles validation and presentation. The method is suitable as a baseline for vocabulary assistance and sentence completion exercises. Future work may compare checkpoints, evaluate a larger sentence set and extend the design to multi-token completion using an approach that models the complete replacement sequence.

7. Reference

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, pp. 4171-4186, 2019.
https://aclanthology.org/N19-1423/.

[2] Hugging Face, "google-bert/bert-base-uncased," model card. Accessed: October 12, 2026.
https://huggingface.co/google-bert/bert-base-uncased.

[3] Hugging Face, "Pipelines," *Transformers Documentation*. Accessed: October 12, 2026.
https://huggingface.co/docs/transformers/main/en/main_classes/pipelines.

[4] Hugging Face, "FillMaskPipeline," *Transformers source code*. Accessed: October 12, 2026.
https://github.com/huggingface/transformers/blob/main/src/transformers/pipelines/fill_mask.py.

[5] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
https://arxiv.org/abs/1706.03762.

Project implementation

Source code, setup instructions and tests:
https://github.com/Katori-bo/bert-fill-mask-assistant